$ ./leitor-exibidor examples/tableswitch.class

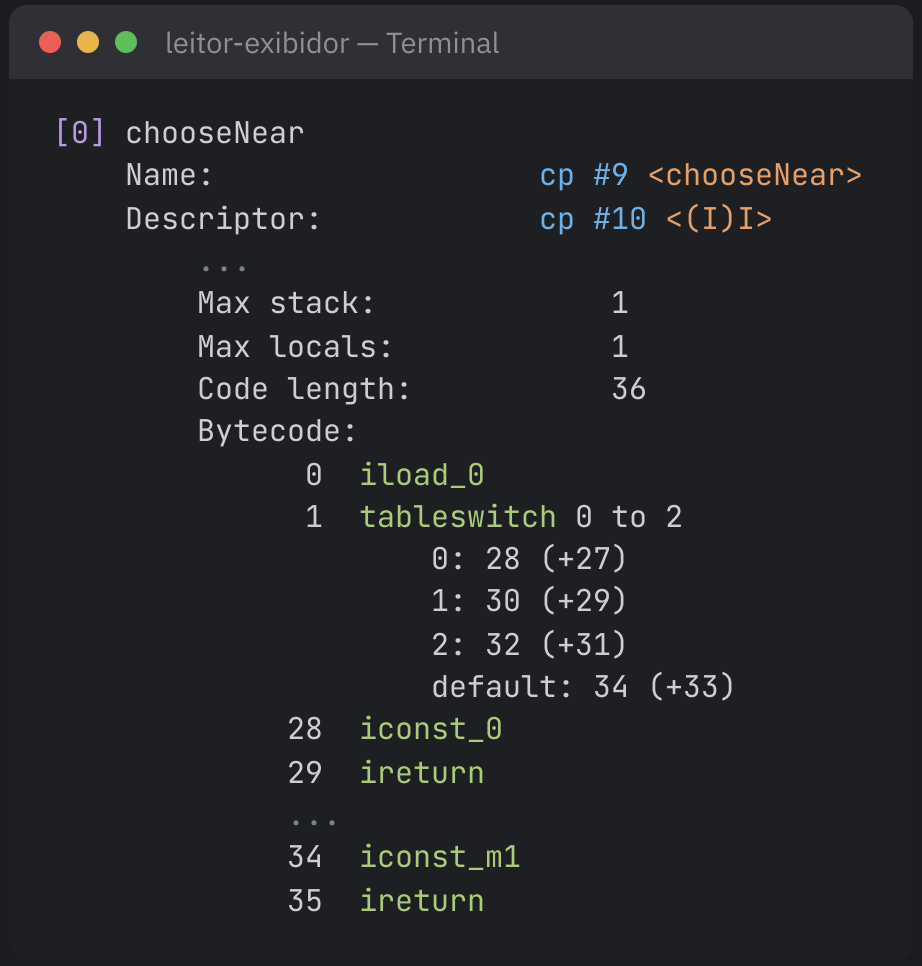
Leitor-exibidor de arquivos .class

Software Básico, UnB

C99 · GCC · formato inspirado no jclasslib

Antonio Coelho · Daniel Morais · Gabriel Pinto · Leonardo Mileo · Lucas Mendes

github.com/LucasMABF/SB



USO

Como compilar e executar

Compilar (GCC, C99)

```
make
# ou, sem o Make:
gcc -std=c99 -Wall -Wextra -pedantic \
  -O2 -g src/*.c -o leitor-exibidor
```

Opções: **-std=c99 -Wall -Wextra -pedantic -O2 -g**. Compila sem nenhum warning.

No Windows: **mingw32-make**, e o executável é **leitor-exibidor.exe**.

Executar

```
./leitor-exibidor X.class
./leitor-exibidor X.class > saida.txt
```

Exibe todas as seções do .class: informações gerais, pool de constantes, interfaces, fields, métodos e atributos. Para salvar em arquivo, basta redirecionar a saída.

Código de saída **0** em caso de sucesso e **1** em caso de erro (arquivo inexistente, magic number inválido, arquivo truncado).

make test roda em todas as classes de teste e salva as saídas em out/; **make asan**, **make cppcheck** e **make fuzz** rodam os analisadores.

CÓDIGO

Organização do código

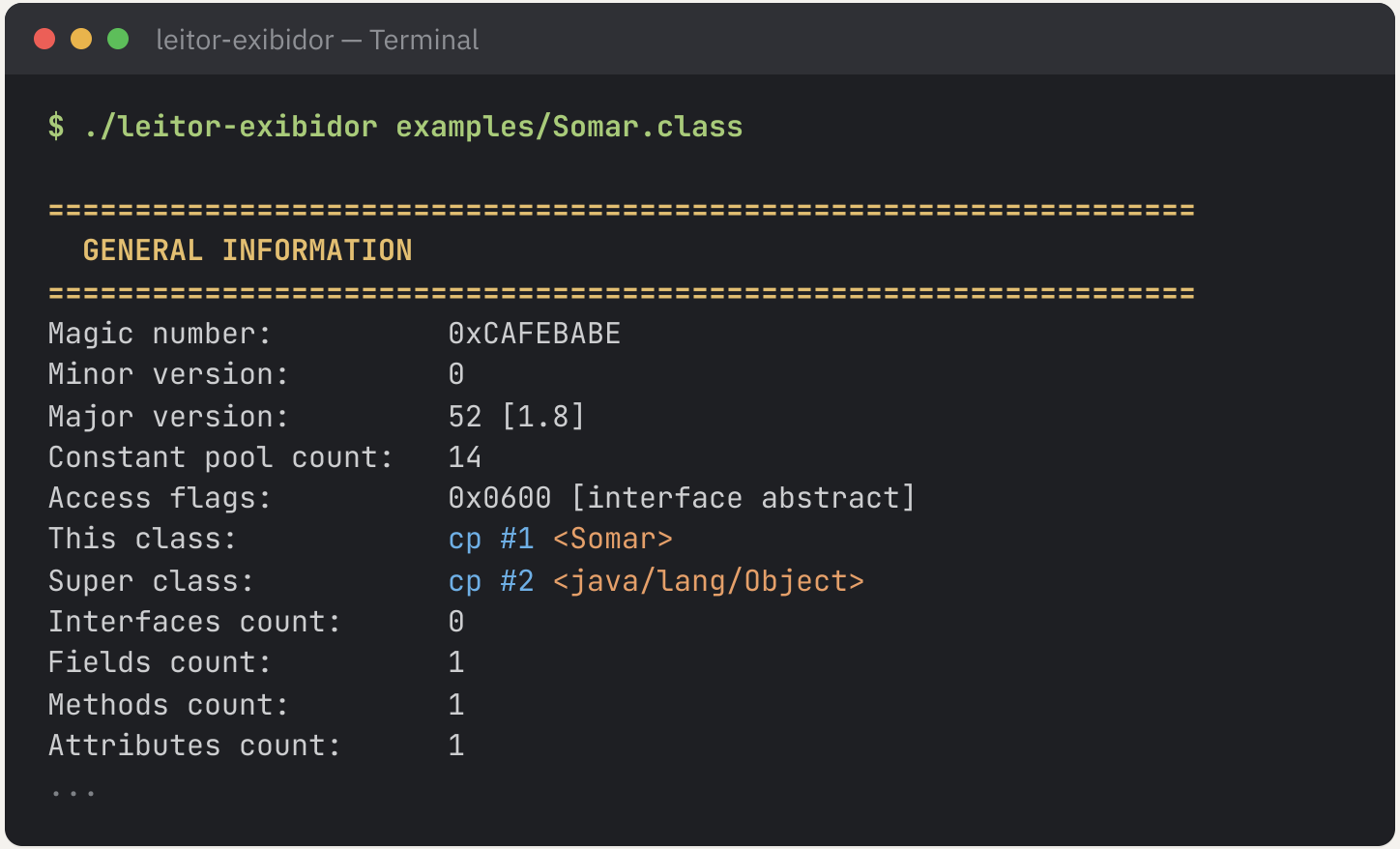
Arquivo	Responsabilidade
reader.c	Leitura big-endian com checagem de limites
classfile.c	Estruturas do .class e parser (JVMS §4)
constant_pool.c	Resolução e exibição do pool de constantes
opcodes.c · bytecode.c	Tabela de opcodes e disassembler do atributo Code
attributes.c	Despacho por nome e decodificadores dos atributos; os demais viram dump hexadecimal
access_flags.c	Flags por contexto (classe, field, método)
printer.c · util.c · main.c	Seções da saída, indentação e dump hexadecimal, ponto de entrada

Os atributos ficam como bytes brutos após o parse e só são decodificados na exibição: um atributo desconhecido nunca quebra a saída.

SOMAR.CLASS

Informações gerais

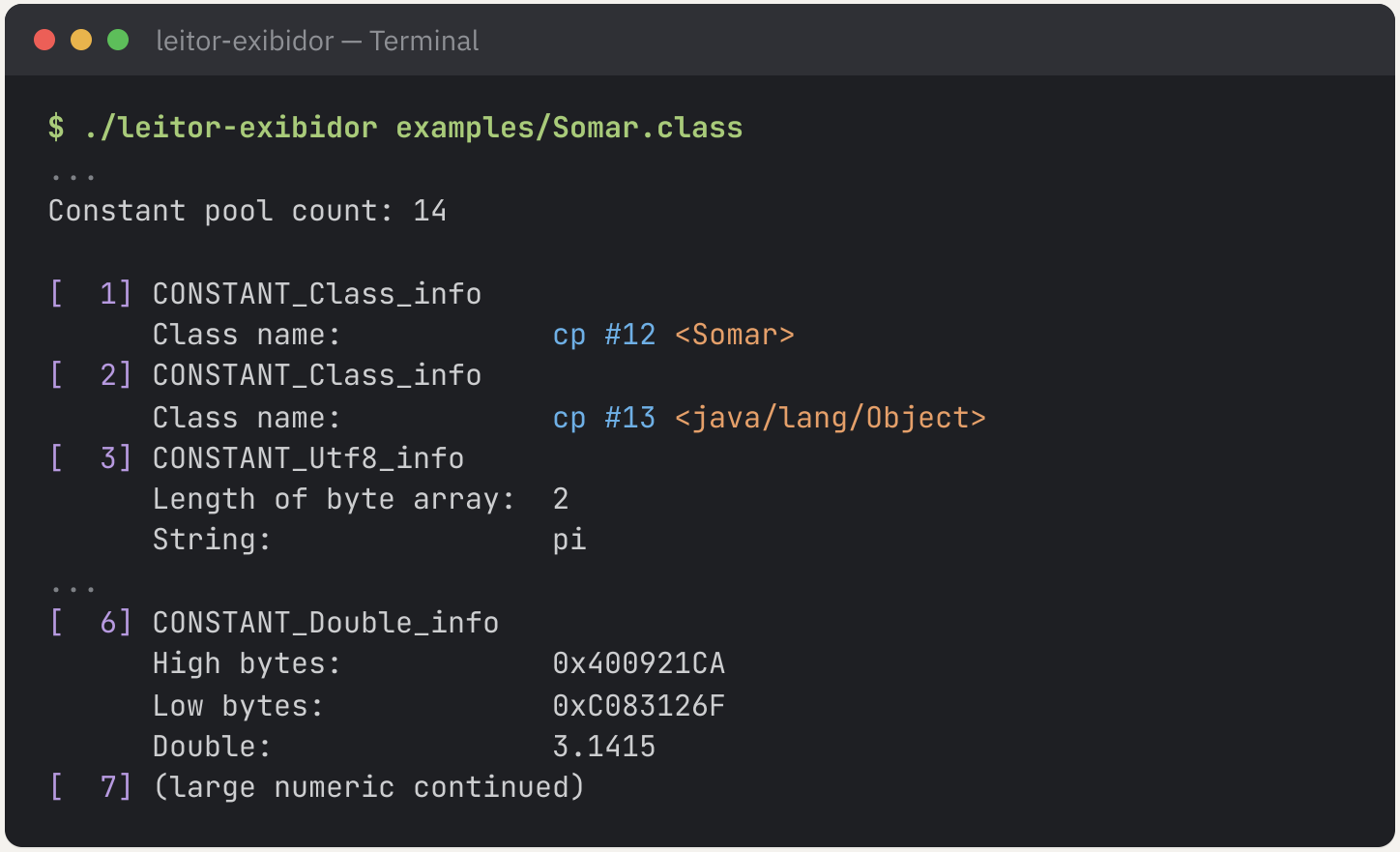
- Magic number e versões, com o nome da versão do Java (52 → 1.8)
- Access flags em hexadecimal e decodificadas
- This e super class com o índice no pool e o nome resolvido



SOMAR.CLASS

Pool de constantes

- Cada entrada com o índice [n] e a tag, como no jclasslib
- Referências mostram cp #índice e a constante resolvida entre < >
- Double e Long mostram high/low bytes e ocupam dois slots
- Strings com \n, \t, \r e o caractere nulo são exibidas com escape



SOMA_CERTO.CLASS · SOMAR.CLASS

Interfaces e fields

- Interfaces: índice no pool e nome da interface
- Fields: nome, descritor e flags do contexto de field
- Atributos de cada field, como o ConstantValue de pi

leitor-exibidor — Terminal

```
$ ./leitor-exibidor examples/soma_certo.class
...
Interfaces count: 1
[0] Interface:          cp #4 <Somar>
```

leitor-exibidor — Terminal

```
$ ./leitor-exibidor examples/Somar.class
...
Fields count: 1

[0] pi
    Name:                   cp #3 <pi>
    Descriptor:             cp #4 <D>
    Access flags:           0x0019 [public static final]
    Attributes count:       1
    [0] ConstantValue
        Attribute name index:  cp #5 <ConstantValue>
        Attribute length:      2
        Constant value index:  cp #6 <3.1415>
```

TABLESWITCH.CLASS

Métodos e bytecode

- Atributo Code: max stack, max locals, tamanho do código e o bytecode
- Cada instrução com o pc e o mnemônico
- tableswitch: destino e offset de cada caso; o padding conta a partir do início do código

leitor-exibidor — Terminal

```
$ ./leitor-exibidor examples/tableswitch.class
...
[0] chooseNear
    Name:                   cp #9 <chooseNear>
    Descriptor:             cp #10 <(I)I>
    Access flags:           0x0009 [public static]
    Attributes count:       1
    [0] Code
        Attribute name index:  cp #11 <Code>
        Attribute length:      76
        Max stack:             1
        Max locals:            1
        Code length:           36
        Bytecode:
              0  iload_0
              1  tableswitch 0 to 2
                     0: 28 (+27)
                     1: 30 (+29)
                     2: 32 (+31)
                     default: 34 (+33)
             28  iconst_0
             29  ireturn
             ...
             34  iconst_m1
             35  ireturn
        Exception table length: 0
```

LOOKUPSWITCH.CLASS

Bytecode: lookupswitch

- Número de pares e cada par chave → destino
- Mesma regra de padding do tableswitch
- Destino absoluto e offset relativo ao pc da instrução

leitor-exibidor — Terminal

```
$ ./leitor-exibidor examples/lookupswitch.class
...
[0] chooseFar
    Name:                   cp #9 <chooseFar>
    Descriptor:             cp #10 <(I)I>
        ...
        Bytecode:
              0  iload_0
              1  lookupswitch 3
                       -100: 36 (+35)
                       0: 38 (+37)
                       100: 40 (+39)
                       default: 42 (+41)
             36  iconst_m1
             37  ireturn
             38  iconst_0
             39  ireturn
             40  iconst_1
             41  ireturn
             42  sipush 200
             45  ireturn
        Exception table length: 0
```

BELOTE.CLASS · MAIN

Operandos que apontam para o pool

- new, anewarray, ldc, invokespecial… mostram #índice e o valor
- Methodref resolvido como classe.método : descritor
- Desvios: destino e offset, como if_icmpge 48 (+25) e goto 20 (-25)

leitor-exibidor — Terminal

```
$ ./leitor-exibidor examples/Belote.class
...
[4] main
    Name:                   cp #73 <main>
    Descriptor:             cp #74 <([Ljava/lang/String;)V>
        ...
        Bytecode:
              0  new #43 <Belote>
              3  dup
              4  invokespecial #44 <Belote.<init> : ()V>
              7  astore_1
              8  invokestatic #45 <Belote.printStatus : ()V>
             11  iconst_4
             12  anewarray #36 <Carta>
             ...
             20  iload 6
             22  iconst_4
             23  if_icmpge 48 (+25)
             26  aload 5
             28  iload 6
             30  new #36 <Carta>
             33  dup
             34  iconst_1
             35  ldc #42 <"1">
             37  iconst_1
             38  invokespecial #39 <Carta.<init> : (ILjava/lang/String;I)V>
             41  aastore
             42  iinc 6 by 1
             45  goto 20 (-25)
```

RECURSOS.CLASS

Atributos da classe

- Nome do atributo pelo pool e tamanho em bytes
- Decodificados: Code, SourceFile, ConstantValue, Signature, Exceptions, LineNumberTable e LocalVariableTable
- Os demais aparecem como dump hexadecimal

leitor-exibidor — Terminal (linhas longas abreviadas com ...)

```
$ ./leitor-exibidor tests/classes/Recursos.class
...
Attributes count:      4
[0] Signature
    Attribute name index:  cp #128 <Signature>
    Attribute length:      2
    Signature index:       cp #176 <<T::Ljava/lang/Comparable<TT;>;>…>
[1] SourceFile
    Attribute name index:  cp #177 <SourceFile>
    Attribute length:      2
    Source file name:      cp #178 <Recursos.java>
[2] BootstrapMethods
    Attribute name index:  cp #179 <BootstrapMethods>
    Attribute length:      22
    (atributo não decodificado) -- dump hexadecimal:
      0000:  00 02 00 BC 00 03 00 B4 00 B5 00 B4 00 BC 00 03
      0010:  00 B8 00 B9 00 B8
[3] InnerClasses
    Attribute name index:  cp #195 <InnerClasses>
    Attribute length:      34
    (atributo não decodificado) -- dump hexadecimal:
      0000:  00 04 00 5A 00 00 00 00 00 00 00 C4 00 0F 00 C6
      0010:  00 08 00 C7 00 0F 00 C9 00 00 00 CA 00 CC 00 CE
      0020:  00 19
```

CAFEBABE.CLASS

Arquivo que não é um .class

leitor-exibidor — Terminal

```
$ ./leitor-exibidor examples/cafebabe.class
Erro ao ler 'examples/cafebabe.class': magic number inválido (0x4D5A9000, esperado 0xCAFEBABE): não é um arquivo
.class
$ echo $?
1
```

- Os 4 primeiros bytes são 4D 5A 90 00 (“MZ”, o cabeçalho de um executável do Windows), não CA FE BA BE.
- O leitor para na verificação do magic number e sai com código 1; a mensagem vai para stderr.
- Arquivos truncados ou corrompidos também geram uma mensagem clara: mais de 2.000 variações testadas, nenhuma derrubou o programa.

ALÉM DOS EXEMPLOS DA DISCIPLINA

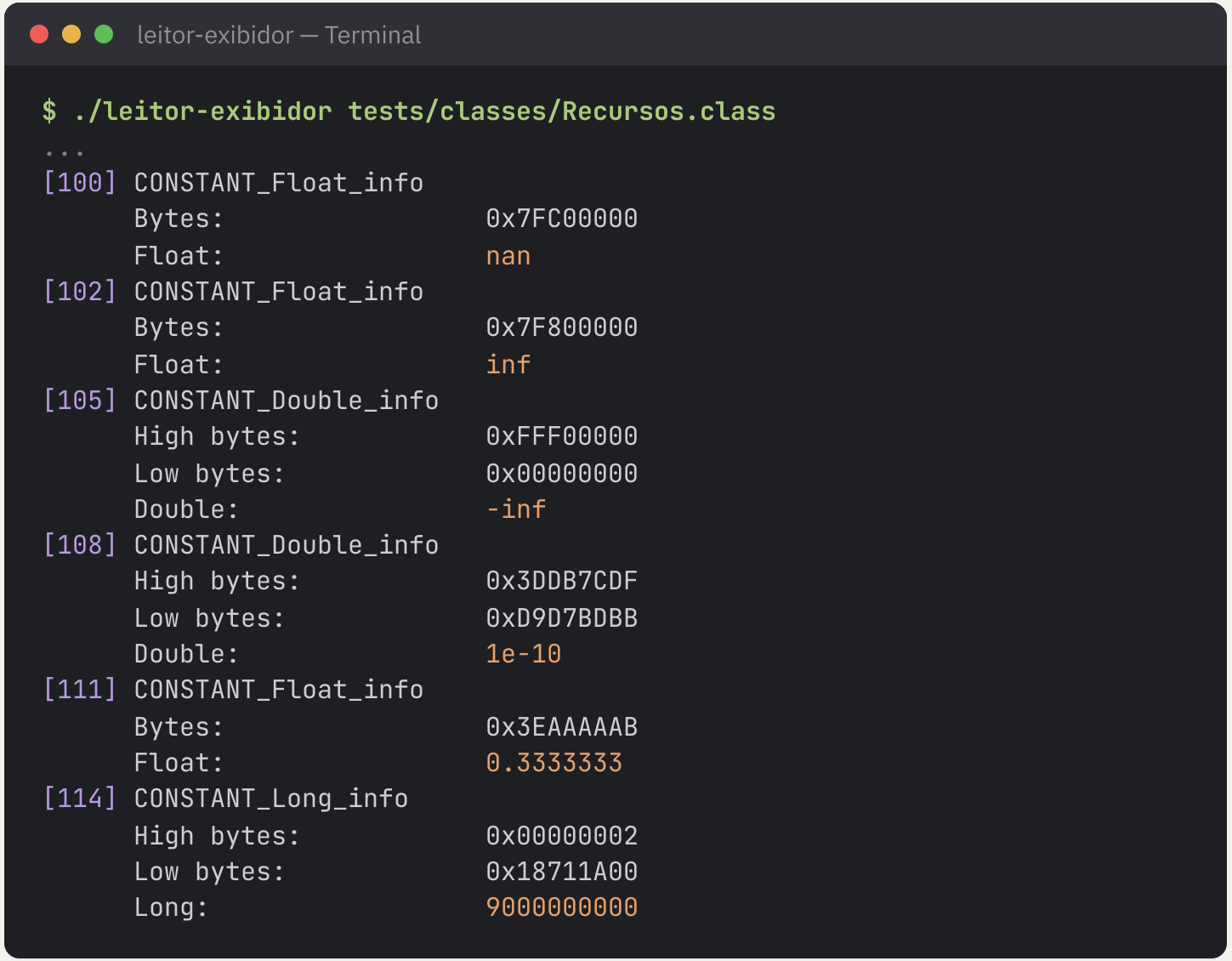
24 classes conferidas com javap -v -p: mesmas constantes, mesmas instruções, mesmos destinos de switch.

Os 19 exemplos da disciplina (o cafebabe.class é rejeitado, como deve ser) e 5 classes extras em tests/java, compiladas com --release 8, que cobrem o que os exemplos não cobrem: wide, NaN e Infinity, exceções, lambdas e classes internas.

RECURSOS.CLASS

Constantes numéricas especiais

- Float, Double e Long mostram os bytes do arquivo e o valor
- O valor é montado a partir dos bits (IEEE 754), incluindo NaN e infinitos
- Float com 7 algarismos significativos e Double com 15: 0.3333333, 1e-10



RECURSOS.CLASS · TENTAR() COM TRY / CATCH / FINALLY

Tabela de exceções e atributos do Code

Cada entrada da tabela aponta para o tipo capturado no pool (cp #0 = any, o finally). LineNumberTable e LocalVariableTable são decodificadas; a StackMapTable aparece em hexadecimal.

leitor-exibidor — Terminal

```
$ ./leitor-exibidor tests/classes/Recursos.class
...
[4] tentar
        ...
        Exception table length: 4
          Nr.  Start PC End PC   Handler  Catch type
          0    0        5        17       cp #60 <java/lang/NumberFormatException>
          1    0        5        32       cp #0 <any>
          2    17       20       32       cp #0 <any>
          3    32       34       32       cp #0 <any>
        ...
        [1] LocalVariableTable
            ...
            Local variable table length: 3
              Nr.  Start PC  Length  Index  Name                    Descriptor
              0    18        14      2      cp #158 <e>             cp #159 <Ljava/lang/NumberFormatException;>
              1    0         47      0      cp #135 <this>          cp #136 <LRecursos;>
              2    0         47      1      cp #160 <s>             cp #119 <Ljava/lang/String;>
        ...
        [3] StackMapTable
            ...
            (atributo não decodificado) -- dump hexadecimal:
              0000:  00 02 51 07 00 3C 4E 07 00 A1
```

LARGO.CLASS

Instrução wide

- Método com 300 variáveis locais (Max locals: 300)
- A partir do índice 256 o javac usa wide istore / wide iload
- wide iinc 299 by 1000: índice e constante de 16 bits

leitor-exibidor — Terminal

```
$ ./leitor-exibidor tests/classes/Largo.class
...
[1] muitasVariaveis
        ...
        Max locals:             300
        Code length:            1463
        Bytecode:
           ...
           1130  istore 253
           1132  sipush 254
           1135  istore 254
           1137  sipush 255
           1140  istore 255
           1142  sipush 256
           1145  wide istore 256
           1149  sipush 257
           1152  wide istore 257
           ...
           1443  sipush 299
           1446  wide istore 299
           1450  wide iinc 299 by 1000
           1456  iload_0
           1457  wide iload 299
           1461  iadd
           1462  ireturn
```

RESUMO

O que foi e o que não foi implementado

Implementado

- Todas as tags do pool, com índice e valor resolvido
- 202 opcodes; switches, wide e operandos do pool resolvidos
- Atributos Code (com a tabela de exceções), SourceFile, ConstantValue, Signature, Exceptions, LineNumberTable e LocalVariableTable
- Nome da versão do Java; escape de \n, \t, \r e do caractere nulo
- Erro claro para arquivo inválido, com código de saída 1
- Saída igual à do javap em 24 classes; mais de 2.000 arquivos corrompidos sem travar
- make test, asan, cppcheck e fuzz

Não implementado

- Demais atributos (StackMapTable, InnerClasses, BootstrapMethods, anotações): aparecem em hexadecimal
- Caracteres fora do BMP (ex.: emoji) em strings Modified UTF-8
- Interface gráfica em árvore como a do jclasslib: a saída é em texto

GRUPO

Divisão de tarefas

Integrante	Matrícula	Parte	Arquivos
Lucas Mendes	241020750	Coordenação, esqueleto, parser, seções gerais, analisadores e entrega	reader.c, classfile.c, printer.c, main.c, Makefile, fuzz/
Antonio Coelho	241038100	Pool de constantes	constant_pool.c
Gabriel Pinto	241002331	Bytecode	opcodes.c, bytecode.c
Leonardo Mileo	241032690	Atributos	attributes.c

Detalhes de cada membro em docs/ATIVIDADES.md. Código: github.com/LucasMABF/SB